

초안 — 검토용

VIGENT 신규 팀원 온보딩

버전 v0.9

생성일 2026-07-27

대외비 — 외부 배포 금지

목차

신호등 범례 — ● [완료] 실측·완료 · ● [진행중] 부분·진행 · ● [미착수] 미착수·미검증 (색이 안 보이면 대괄호 텍스트로 판단)

1. 00. 시작하기 전에 (3분 요약)
2. 40. Claude Code 사용법 ★ 바이브코딩 규칙
3. 50. 완전초보 온보딩 (가장 자세하게)
4. 60. 협업 규칙 — Git
5. 70. 역할 분담과 주간 루틴
6. 80. 용어 사전

※ 정확한 페이지 번호는 PDF 뷰어의 페이지 표시를 참고하세요. (브라우저 인쇄 시 하단 페이지번호 옵션 사용 가능)

00. 시작하기 전에 (3분 요약)

이 문서는 팀원 전원(대표 포함)이 가장 먼저 읽는 문서입니다. 바이브코딩·팀 프로젝트가 처음이어도 괜찮습니다.

VIGENT가 뭘 만드는 회사인가 (5줄)

- VIGENT는 공장·건설현장의 CCTV 영상을 AI가 보고 위험을 잡아내는 산업안전 플랫폼입니다.
- 카메라가 "위험구역에 사람이 들어왔다 / 보호구를 안 썼다 / 불·연기가 났다"를 자동으로 감지해 경보합니다. (이것을 비전(Vision, L1)이라 부릅니다.)
- 감지 결과로 위험성평가서 같은 안전 서류를 AI가 자동 작성합니다. (이것을 에이전트(Agent, L2)라 부릅니다.)
- 즉 "보는 눈(비전) + 판단하는 머리(에이전트)"가 한 시스템으로 붙어 있습니다.
- VIGENT는 보조 감시 도구입니다 — 법으로 정해진 안전장치(방호장치 등)를 대체하지 않고, 추가로 지켜보는 신호를 줍니다.

지금 어디까지 왔나 (3줄)

- 비전·에이전트 코드와 자체 테스트(82개 통과)는 갖춰졌고, 동시부하 크래시(F-14) 같은 큰 안정성 문제는 해결했습니다.
- 하지만 실제 현장 CCTV로 정확도를 쟀 적이 거의 없습니다(실영상 검증 1건). 공개 데이터셋 수치만 있습니다.
- 감지 클래스는 3클래스 가동(person·PPE·fire/smoke) + forklift(지게차)는 재학습 대기입니다.

앞으로 뭘 할 건가 (3줄)

- 비전(L1): 현장 영상으로 정확도(오탐·미탐)를 재고, 부족한 클래스(안전모 미착용 등)를 재학습해 KISA 인증 기준($F1 \geq 0.90$)을 향합니다.
- 에이전트(L2): AI가 만든 안전서류의 품질을 계속하고, 잘못된 법령 인용(환각)을 막고, 비전 감지와 자동 연동합니다.
- 두 트랙이 만나 파일럿(시범 설치) → 유상 납품으로 갑니다. 상세 단계는 10 · 20 문서에 있습니다.

문서 읽는 순서

순서	문서	누가 꼭 읽나
1	00_시작하기_전에 (이 문서)	전원
2	01.현황_숫자로_보기	전원 (특히 대표)
3	10_비전트랙_기획안	비전 담당·대표
3	20_에이전트트랙_기획안	에이전트 담당·대표
4	30_통합_로드맵	전원
5	40_클로드코드_사용법	전원 (개발 참여자 필수)
6	50_완전초보_온보딩	신규 팀원 필수 (설치 처음이면)
7	60_협업규칙_.git	코드 만지는 전원
8	70_역할분담과_주간루틴	전원
9	80_용어사전	모르는 단어 나올 때마다

- 개발이 처음이면: 00 → 01 → 50(온보딩) → 40 → 60 순서를 권장합니다.
- 대표·사업 담당이면: 00 → 01 → 10 → 20 → 30 을 보면 전체 그림과 일정이 잡힙니다.

⚠ 보안 주의 (반드시)

- 이 리포(코드·수치·문서)의 내용을 외부에 공유하지 마세요. 대외비입니다.
- 공개 시연·외부 발표 전에는 반드시 특허 출원을 먼저 합니다. (특히 RIG 모듈은 특허 대상이라 세부 동작은 팀 내에서도 [대외비] 로 다룹니다.)
- .env (비밀키)·모델 가중치·현장 영상은 절대 git에 올리지 않습니다 (상세: 60_협업규칙_.git.md).

개발 도구는 Claude Code 하나로 통일합니다. 다른 IDE·플러그인은 쓰지 않습니다 (이유·사용법: 40_클로드코드_사용법.md).

40. Claude Code 사용법 ★ 바이브코딩 규칙

우리 팀은 Claude Code 하나만 씁니다. 이유: 도구가 여러 개면 팀원마다 환경·습관이 갈려 협업이 깨집니다. 하나로 통일하면 지시문·문제해결 방식을 공유할 수 있습니다.

설치·실행 (화면 흐름)

- 터미널을 엽니다 (macOS: Spotlight에서 "터미널" / Windows: PowerShell). 상세는 `50_완전초보_온보딩.md`.
- 프로젝트 폴더로 이동합니다.

```
cd ~/Desktop/VIGENT
```

실행하면 이런 게 보입니다:

(아무 메시지 없이 프롬프트가 VIGENT 폴더로 바뀝니다)

- Claude Code를 실행합니다.

```
claude
```

실행하면 이런 게 보입니다:

```
Claude Code
> (여기에 지시를 입력)
```

좋은 지시문 4원칙

- 목적 먼저 — "무엇을 왜" 한 줄로.
- 건드릴 파일 범위 명시 — 어느 파일까지 만져도 되는지.
- 완료 조건 제시 — 무엇이 되면 끝인지(숫자로).
- 한 번에 하나 — 여러 개를 한꺼번에 시키지 않기.

나쁜 지시문 vs 좋은 지시문 (이 프로젝트 실제 작업으로)

예시 1 — 정확도 측정

- 나쁨: "정확도 좀 봐줘"
- 좋음: "person-PPE-fire/smoke 클래스별 precision-recall-F1을 산출하는 스크립트를 `tools/`에 만들어줘. 기존 `benchmarks/` 코드를 재활용하고, 측정 환경(MPS/CPU)을 리포트에 병기해줘. 코드 수정 전 계획 먼저 보여줘." (V-02 티켓)

예시 2 — 추적기 교체

- 나쁨: "추적 더 좋게 바꿔"
- 좋음: "추적기 A/B 결과 SORT가 단편화가 심해. `rfdetr_service.py`·`rfdetr_zone_track.py`의 SORTTracker를 ByteTrackTracker로 교체하는 계획을 먼저 보여주고, 게이트(테스트 82) 통과 확인 후 커밋해줘. 다른 파일은 건드리지 마."

예시 3 — 연동 배선

- 나쁨: "감지된 거 서류에 자동으로 넣어줘"
- 좋음: "`data_engine.log_event` → `scribe.build_checklist` 경로를 조사해줘. 지금은 빈도만 재사용하는데, 감지 규칙이 체크리스트 항목을 자동 X로 반영하도록 바꾸는 설계를 **계획으로** 먼저 보여줘. 미감지 항목은 '수동확인' 유지. `golden_regression` 회귀 없게." (A-01 티켓)

작업 흐름 (반드시 이 순서)

① 지시 입력 → ② "계획 먼저 보여줘" 요청 → ③ 계획 확인 후 "진행해" 승인
→ ④ 결과·게이트 확인 → ⑤ 커밋

- **핵심:** 코드를 바꾸는 지시는 **항상 계획을 먼저 받고** 승인하세요. 바로 실행시키지 않습니다.

CLAUDE.md 가 뭔가

- 프로젝트 루트의 `CLAUDE.md` 는 **Claude Code가 매 대화마다 자동으로 읽는 규칙서**입니다.
- 여기 규칙을 적어두면 모든 팀원의 Claude Code가 같은 규칙으로 동작합니다.
- 규칙 추가법: "CLAUDE.md에 '~~ 규칙을 추가해줘'"라고 지시하면 됩니다. (단, 팀 합의 후)

세션 관리

- 대화가 길어지면 **한 작업 끝 → 커밋 → 새 대화 시작**. 짧게 끊어야 Claude가 헛갈리지 않습니다.
- **작업 단위로 커밋**하세요. 한 커밋에 여러 작업을 몰지 마세요.

절대 하지 말 것

- ❌ 확인 없이 대량 파일 수정을 승인하기
- ❌ 이해 못 한 코드를 커밋하기 (뭐 하는지 물어보고 이해한 뒤 커밋)
- ❌ 에러를 숨기고 다음 단계로 넘어가기 (에러는 반드시 해결·공유)

막혔을 때 3단계

1. 에러 메시지 전문을 복사 (일부만 말고 전체).
2. Claude Code에 붙여넣기 + "이 에러가 났어. 원인과 해결을 알려줘".
3. 그래도 안 되면 팀 채널에 에러 전문 공유.

50. 완전초보 온보딩 (가장 자세하게)

컴퓨터에 아무것도 안 깔린 상태를 가정합니다. macOS와 Windows를 나눠서 씁니다. 상세 개발 환경은 `docs/ONBOARDING.md` 도 함께 참고합니다(이 문서는 그 초보자용 버전입니다).

❌ 이거 없으면 아무것도 안 됩니다 (제일 먼저)

git으로 코드를 받아도 아래 2개는 안 따라옵니다. 반드시 팀에서 따로 받아 넣으세요.

① .env (비밀키 파일)

- git에 올리면 안 되는 API 키가 들어갑니다 → **각자 만들어야** 합니다.

```
cp .env.example .env
```

실행하면 이런 게 보입니다:

(아무 메시지 없이 .env 파일이 생깁니다)

- 그다음 .env 를 열어 팀에게 받은 키를 채웁니다(예: `OPENAI_API_KEY`). 키는 절대 채탕·git에 쓰지 마세요.

② 모델 가중치 `weights/` (812MB, 18개 파일)

- 용량이 커서 git에 없습니다. 없으면 검출이 엉뚱하게 동작(COCO 풀백)해 정확도가 급락합니다(과거 F-8 사고).
- 팀 공유 위치(클라우드 드라이브/USB)에서 받아 `vigent-core/weights/` 폴더에 넣습니다.
- 확인:

```
ls vigent-core/weights/ | wc -l
```

실행하면 이런 게 보입니다:

18

[스크린샷 자리] weights 폴더에 파일 18개가 들어있는 모습

이 2개가 준비되기 전에는 서버가 제대로 안 돛니다. 아래 설치를 먼저 하고, 이 2개로 마무리하세요.

macOS

1. 터미널 열기

- `Cmd(⌘) + Space` → "터미널" 입력 → Enter.

[스크린샷 자리]

2. 내 위치 확인·이동·목록

```
pwd          # 지금 어느 폴더인지
cd ~/Desktop/VIGENT  # VIGENT 폴더로 이동
ls           # 폴더 안 목록 보기
```

실행하면 이런 게 보입니다:

```
/Users/본인이름/Desktop/VIGENT
CLAUDE.md  README.md  vigent-core  docs  ...  VIGENT 대외비 · 초안—검토용 v0.9
```

3. Git 확인·설치·설정

```
git --version          # 설치돼 있으면 버전이 뜸
```

- 안 깔렸으면: `xcode-select --install` 실행 → 설치 창을 따라갑니다.
- 최초 1회 설정:

```
git config --global user.name "본인이름"  
git config --global user.email "본인이메일"
```

4. Python·가상환경·의존성

```
python3 --version          # 3.11~3.13 권장  
python3 -m venv .venv      # 가상환경 만들기  
source .venv/bin/activate  # 활성화 (앞에 (.venv) 가 붙음)  
pip install -r requirements.txt # 의존성 설치
```

- opencv 정리(중요, docs/ONBOARDING.md \$2.2와 동일):

```
pip uninstall -y opencv-python opencv-contrib-python  
pip install --force-reinstall --no-deps opencv-contrib-python-headless==4.13.0.92
```

5. 리포 내려받기 (처음 한 번만)

```
git clone <여기에 저장소 주소> VIGENT  
cd VIGENT
```

6. Claude Code 설치·실행

```
claude          # 프로젝트 폴더에서 실행
```

[스크린샷 자리] Claude Code 시작 화면

7. 첫 실행 성공 확인

- 서버 켜기:

```
./run.sh
```

- 다른 터미널에서:

```
curl http://127.0.0.1:8010/health
```

실행하면 이런 게 보이면 성공입니다:

```
{"status":"ok","loaded":true, ... }
```

- `"loaded":true` 가 보이면 모델이 제대로 올라간 것입니다. (안 뜨면 \$가중치 확인)

Windows

⚠ 이 프로젝트는 macOS에서 개발됐습니다. Windows에서 처음 켤 때 잔가지 조정이 필요할 수 있습니다. NVIDIA GPU가 있으면 검출이 더 빠르고 재학습도 가능합니다(상세 docs/INFRA_REQUIREMENTS.md).

1. 터미널(PowerShell) 열기

VIGENT 대외비 · 초안—검토용 v0.9

- 시작 버튼 → "PowerShell" 입력 → Enter.

[스크린샷 자리]

2. 내 위치 확인·이동·목록

```
Get-Location          # 지금 폴더 (pwd 대신)
cd $HOME\Desktop\VIGENT # 이동
dir                   # 목록 (ls 대신 dir)
```

3. Git 확인·설치·설정

```
git --version
```

- 안 깔렸으면: <https://git-scm.com> 에서 설치.

```
git config --global user.name "본인이름"
git config --global user.email "본인이메일"
```

4. Python·가상환경·의존성

```
python --version          # 3.11~3.13
python -m venv .venv
.venv\Scripts\activate    # 활성화 (macOS와 경로 다름)
pip install -r requirements.txt
```

- ⚠ 로컬 VLM(mlx)은 애플 전용이라 Windows에서 설치·실행 안 됩니다. requirements-optional.txt 의 mlx 줄은 건너뛰고, VLM은 클라우드 (OPENAI_API_KEY + .env 의 VIGENT_CLOUD_VLM=1)로 씁니다.
- ⚠ NVIDIA GPU면 torch를 CUDA용으로 따로 설치하세요(자세히는 docs/INFRA_REQUIREMENTS.md).

5·6·7. clone·Claude·실행

```
git clone <여기에 저장소 주소> VIGENT
cd VIGENT
claude
```

- 서버 실행 후 확인:

```
curl http://127.0.0.1:8010/health
```

"loaded":true 가 보이면 성공.

자주 나는 에러 5가지와 대처

증상	원인	대처
Permission denied	실행 권한 없음	chmod +x run.sh (mac) / 관리자 PowerShell
command not found: claude / python	미설치·경로 문제	재설치, 터미널 새로 열기
Address already in use (포트 8010)	이미 서버가 떠 있음	기존 프로세스 종료 후 재실행
ModuleNotFoundError	가상환경 미활성	source .venv/bin/activate (mac)/ .venv\Scripts\activate (win)
검출이 안 됨 / "loaded":false	모델 가중치 없음	vigent-core/weights/ 에 812MB 복사(맨 위 ② 참고)

막혔을 때 행동 순서

1. 에러 메시지 전체를 복사한다 (일부만 X).
2. Claude Code에 "이 에러가 났어" + 붙여넣기 → 원인·해결 요청.
3. 그래도 안 되면 팀 채널에 에러 전문을 공유한다.

VIGENT 대외비 · 초안—검토용 v0.9

60. 협업 규칙 — Git

깃(git)이 뭔가 (비유)

- 게임 세이브 파일과 같습니다. 작업을 "저장(커밋)"하면 언제든지 그 지점으로 돌아갈 수 있습니다.
- 여러 명이 각자 세이브를 만들고 나중에 합치는(merge) 도구입니다. 문서 버전관리(v1, v2 ...)를 자동으로 해준다고 보면 됩니다.

만드시 지킬 5가지 규칙 (제일 중요)

1. 작업 전 `git pull` — 남이 올린 최신을 먼저 받는다.
2. 내 작업은 내 브랜치에서 — `main` 에 직접 작업하지 않는다.
3. 작업 단위로 커밋 — 한 번에 하나. 메시지에 무엇을 했는지 쓴다.
4. `.env`·가중치·현장영상은 절대 올리지 않는다.
5. 막히면 멈추고 공유 — 특히 충돌(conflict)은 혼자 해결하려 하지 말 것.

하루 작업 흐름 (명령어 + 한 줄 해설)

```
git pull                # 최신 내용 받기
git checkout -b feat/내작업이름 # 내 작업 공간(브랜치) 만들기
# ... 작업 ...
git status              # 뭐가 바뀌었는지 보기
git add -A              # 바뀐 것 전부 담기
git commit -m "feat: 무엇을 했는지" # 저장(커밋)하기
git push -u origin feat/내작업이름 # 내 브랜치를 서버에 올리기
```

브랜치 이름 규칙

- 형식: 종류/짧은설명 — 예: `feat/f1-측정`, `fix/포트충돌`, `docs/온보딩보완`

커밋 메시지 규칙 (종류 + 한글 설명)

- `feat`: 새 기능 / `fix`: 버그 수정 / `docs`: 문서 / `test`: 테스트 / `refactor`: 구조 개선
- 예시 5개:
 - `feat`: 클래스별 F1 산출 스크립트 추가 (V-02)
 - `fix`: 가중치 부재 시 fail-fast 전환 (F-8)
 - `docs`: 신규 팀원 온보딩 문서 추가
 - `test`: rig_monitor 상태전이 테스트 보강
 - `refactor`: 이벤트-체크리스트 연동 구조 정리 (A-01)

절대 치면 안 되는 명령어와 이유

명령어	왜 위험한가
<code>git push --force</code>	남의 작업을 덮어써 삭제할 수 있음
<code>git reset --hard</code>	저장 안 한 내 작업이 되돌릴 수 없이 사라짐
<code>rm -rf</code>	폴더·파일을 복구 불가능하게 삭제
main에 직접 push	리뷰 없이 전원에게 영향, 사고 시 추적 어려움

충돌(conflict)이 났을 때 (초보자)

1. 당황하지 마세요. 흔한 일입니다.
2. 아무것도 저장하지 말고 상황 그대로 둡니다.
3. 화면(충돌 메시지)을 Claude Code에 그대로 붙여넣고 "충돌이 났어. 어떻게 해결하는지 알려줘"라고 물어봅니다.

git에 올리면 안 되는 것 (다시)

- `.env`, API 키·비밀번호
- 모델 가중치(`weights/`, 812MB)
- 현장 영상·개인정보 포함 파일

- (이미 `.gitignore` 가 대부분 막아두지만, 새 파일 추가 시 주의)

코드 리뷰 (PR)

- 작업을 올린 뒤 GitHub에서 **PR(Pull Request, 병합 요청)**을 만듭니다.
- 리뷰어를 지정합니다(내 폴더 담당 + 공유부면 양쪽 — `70_역할분담` 참고).
- 승인 없이 merge 금지. 최소 1명 승인 후 합칩니다.

70. 역할 분담과 주간 루틴

역할과 담당 폴더 (파일 충돌을 구조적으로 예방)

감사 결과 비전·에이전트는 서로 import하지 않아(느슨한 결합) 폴더로 나누면 충돌이 거의 없습니다.

역할	담당 폴더/파일	대표 티켓
비전 (L1)	vigent-core/rfdetr_service.py · worker.py · proximity.py · ergonomics.py · zone_*.py · routers/{detect,zone,ppe}.py · weights/	V-01~V-08
에이전트 (L2)	vigent-core/agents/ · routers/{safety_core,tbm,incident}.py · eval/golden/ · config/corpus/	A-01~A-08
문서·사업	docs/ · docs/team/ · 제안서·조달	제안·인프라·개입정보
현장 검증	현장 영상 수집·오탐 분류·라벨(데이터, 코드 아님)	V-05·A-05
공유부 (양쪽 리뷰 필수)	app_state.py · web_util.py · main.py · data_engine.py · vision.yaml	—

- 공유부를 바꿀 땐 반드시 상대 역할과 조율하고 PR 리뷰를 양쪽에서 받습니다.
- (참고) CODEOWNERS 파일로 이 규칙을 자동화할 수 있습니다 — GitHub에 올린 뒤 도입 검토.

주 1회 30분 동기화 미팅 — 아젠다 템플릿

- 지난주 완료 티켓 (각자 1분) — 5분
- 병목 3개 (V-01·V-02·A-01) 진행 점검 — 10분
- 교차 의존 막힘 확인 (한쪽 늦으면 양쪽 정지) — 5분
- 이번 주 각자 목표 티켓 1개 선언 — 5분
- 위험 신호(조기 경보) 공유 — 5분

데일리 3줄 보고 양식 (팀 채널)

[이름] YYYY-MM-DD
어제 한 것 : (티켓 번호 + 한 줄)
오늘 할 것 : (티켓 번호 + 한 줄)
막힌 것 : (없으면 '없음')

티켓 진행 상태 관리 — 방식 선택

- A. 문서 체크박스: 이 문서들의 ☐ 체크박스를 직접 채움. 간단하나 여러 명이 동시에 고치면 충돌.
- B. GitHub Issues: 티켓 하나 = 이슈 하나. 담당·상태·댓글이 자동 기록되고 PR과 연결됨.

권장: B(GitHub Issues). 이유 — (1) 충돌 없음(각 이슈 독립), (2) 누가 뭘 하는지 한눈에, (3) PR·커밋과 자동 연결로 이력이 남음. 초보 팀도 "이슈 만들고 → 담당 정하고 → 완료 시 닫기"만 익히면 됩니다.

팀원별 "첫 주에 이것만 하면 성공"

VIGENT 대외비 · 초안—검토용 v0.9

역할	첫 주 목표
비전(L1)	V-01 완료(가중치 공유 경로 확정 + fail-fast) — 팀 확장의 1번 장애물 제거
에이전트(L2)	A-04(Coach 범위 확정) or A-03(글든 1건 추가) 중 1개
문서·사업	제안서(docs/PILOT_PROPOSAL.md) 현장 맞춤 1버전 다듬기
현장검증	현장 영상 확보 경로 1개 확정(카메라·연락처)
전원	50_완전초보_온보딩 따라 서버 /health 에 loaded:true 띄우고 첫 커밋

80. 용어 사전

이 리포에서 실제로 쓰이는 용어만 모았습니다. 각 항목 = 한 줄 정의 + "우리 프로젝트에선 이렇게".

개발·협업

용어	정의	우리 프로젝트에선
커밋(commit)	작업 내용을 한 시점으로 저장하는 것	"작업 단위로 커밋" — 한 번에 하나
브랜치(branch)	내 작업만 담는 분리된 공간	<code>feat/내작업이름</code> 으로 만들어 작업
PR(Pull Request)	내 브랜치를 main에 합쳐달라는 요청	승인 없이 merge 금지
머지(merge)	브랜치를 합치는 것	PR 승인 후에만
게이트(gate)	커밋 전 통과해야 하는 검사	ruff-mypy-테스트 82·OpenAPI 무변경
스텝(stub)	껍데기만 있고 실행 안 하는 코드	Coach 에이전트(12줄)
fail-fast	문제 시 조용히 넘어가지 않고 즉시 멈춤	가중치 없으면 폴백 대신 멈추기(V-01)
폴백(fallback)	실패 시 대체 동작	가중치 없으면 COCO로 폴백(주의: 정확도 ↓, F-8)

비전(L1)

용어	정의	우리 프로젝트에선
mAP	검출 정확도 종합 지표(0~1)	fire/smoke 공개셋 mAP@50 80.13%
precision(정밀도)	잡았다고 한 것 중 맞은 비율(오탐의 반대)	대부분 미측정 — 재측정 필요
recall(재현율)	실제 위험 중 잡아낸 비율(미탐의 반대)	PPE·person 수치는 대부분 recall만
F1	precision과 recall의 조화평균	KISA 기준 F1≥0.90 (현재 미산출, V-02)
추론 지연(latency)	한 프레임 처리에 걸리는 시간(ms)	타일 +415ms/ff(CPU)
RF-DETR	우리가 쓰는 객체 검출 모델	검출 코어(<code>rfdetr_service.py</code>)
포즈 추정(pose)	사람 관절 위치 추정	RTMPose/yolov8n-pose (<code>ergonomics.py</code>)
추적(tracking)	프레임 간 같은 사람에 같은 ID 부여	ByteTrack(SORT에서 교체)
ByteTrack	우리가 채택한 추적 알고리즘	ID 스위치 0으로 개선
타일링	위험구역만 잘라 재검출해 소형작업자를 잡는 기법	recall 0→89~95%(단 +415ms)
MPS	애플 칩(맥)의 GPU 가속	현재 개발환경(배포는 CUDA)
CUDA	NVIDIA GPU 가속	배포·재학습 목표 환경
엣지(edge)/RK3588	현장 저전력 소형 처리장치	포팅 검토 단계(코드 없음)

에이전트(L2)

용어	정의	우리 프로젝트에선
에이전트(agent)	특정 역할을 맡는 AI 모듈	Scribe·Copilot·Guard 등 6개
골든셋(golden set)	정답이 있는 평가용 표본	체크리스트 3건 + 홀드아웃 15케이스
채점 엔진	산출물을 정답과 비교해 점수화	<code>golden_regression.py</code>
베이스라인	회귀(나빠짐) 감지 기준선	품질 합격선 아님 — 정답 확정 아님
하이트리스트	허용된 법령 조문 목록	밖의 조문 인용은 차단(F-11)
환각(hallucination)	AI가 없는 사실·조문을 지어냄	crane 골든서 4건 관찰
상태기계	조건에 따라 상태가 전이되는 로직	RIG 모듈(세부는 대외비)
LLM	대형 언어모델(텍스트 생성 AI)	OpenAI/Anthropic(외부 API)
온프레임(on-prem)	외부 클라우드 없이 자체 장비에서 실행	폐쇄망 대응 과제(현재 미지원)
폐쇄망	인터넷이 차단된 내부망	VIGENT 대외비 · 초안—검토용 v0.9 공장 다수의 조건(S5)

사업·인증

용어	정의	우리 프로젝트에선
파일럿(pilot)	유상 전 시범 설치·검증	위험구역 침입 단일기능부터
KISA 인증	지능형 CCTV 성능 인증(F1≥0.90)	S4 목표
TTA GS	SW 품질 인증 등급	S5(조달 진입)
조달 등록	공공조달 판매 자격	S5